POROJETMIGRATION

# MÉMOIRE DE SESSION

## Continuation du projet dans une nouvelle discussion

Principes de travail, état technique complet, décisions, contrats CRM / Google Sheets / SMS, tests, erreurs à ne pas refaire et actions restant à mener avant production.

| | |
|---|---|
| **Dépôt** | manajahelvin-ux/PorojetMigration |
| **Branche de référence** | main |
| **Dernier commit intégré** | 36567bc — fusion de la PR #11 |
| **Session de génération** | arena/e1f685c7-porojetmigration |
| **Date** | 27 octobre 2026 |
| **PR fusionnées** | #1 à #11 |

### Utilisation

Ouvrir une nouvelle discussion sur le même dépôt, joindre ce PDF ou le fichier Markdown, puis coller le message de reprise de la section 1. Le nouvel assistant doit d'abord vérifier l'état réel du dépôt avant toute modification.

Document généré à partir de MEMOIRE-SESSION-CONTINUATION.md. Le code déjà présent dans main reste la source de vérité technique.

## SOMMAIRE


- 1. MESSAGE À COLLER DANS LE NOUVEAU CHAT
- 2. RÉSUMÉ EXÉCUTIF
- 3. PRINCIPES PERMANENTS À RESPECTER
- 4. ÉTAT FINAL DU DÉPÔT
- 5. HISTORIQUE DES TRAVAUX ET DÉCISIONS
- 6. CONTRAT CRM HELLOLEAD
- 7. CONTRAT GOOGLE SHEETS
- 8. FONCTIONNEMENT DU lead-handler.js
- 9. TRACKING ET CONSENTEMENT
- 10. VÉRIFICATION SMS — CONTRAT COMPLET
- 11. WRAPPER IFRAME ET RELAIS SMS
- 12. IDENTITÉS CRM DES SIX PREMIERS PROJETS
- 13. RÈGLES DE TEST NON NÉGOCIABLES
- 14. ERREURS DÉJÀ RENCONTRÉES À NE PAS REFAIRE
- 15. PROCÉDURE POUR AJOUTER OU TRAITER UNE NOUVELLE LP
- 16. CE QUI RESTE À FAIRE CÔTÉ PRODUCTION
- 17. ARBORESCENCE SIMPLIFIÉE FINALE
- 18. HISTORIQUE GIT RÉSUMÉ
- 19. CONSIGNES POUR LE PROCHAIN ASSISTANT
- 20. PHRASE DE SYNTHÈSE À RETENIR


Le document source Markdown, plus facile à réutiliser par un assistant, accompagne ce PDF sous le nom MEMOIRE-SESSION-CONTINUATION.md.

# 1. MESSAGE À COLLER DANS LE NOUVEAU CHAT

Copier-coller le texte suivant dans la nouvelle discussion, avec ce fichier MD ou le PDF correspondant :

```
Lis entièrement MEMOIRE-SESSION-CONTINUATION.md (ou son PDF) et utilise-le comme mémoire officielle du projet
  PorojetMigration.

Continue exactement avec les mêmes principes et règles que la session précédente :
- agis comme un expert senior, solution rapide et efficace ;
- réponds en français ;
- analyse avant de modifier ;
- préserve l'identité, le contenu et le design de chaque LP ;
- n'écrase jamais une LP avec une autre ;
- applique la parité fonctionnelle avec la LP de référence assurance-consultant-conseil-v2 ;
- la vérification SMS demandée dans la dernière phase fait désormais partie du standard ;
- teste avec jsdom et le vrai jQuery en simulant le chemin Formly ;
- génère les ZIP complets demandés ;
- travaille uniquement sur la branche de session indiquée par Arena ;
- n'ouvre une PR et ne fusionne que si je le demande explicitement ;
- avant toute PR, fais git fetch origin et vérifie si main a été recréée par upload.

Pars de l'état déjà intégré dans main au commit 36567bc. Ne refais pas le travail déjà mergé dans les PR #1 à
  #11. Commence par vérifier git status, la branche, origin/main et les fichiers réellement présents dans le
  dépôt.
```

## 2. RÉSUMÉ EXÉCUTIF

Le projet consistait à rendre autonomes et réellement fonctionnelles des landing pages exportées depuis Webflow puis converties par lp2site. À l'origine, plusieurs formulaires semblaient fonctionner visuellement mais échouaient hors Webflow avec le message « Oops! », envoyaient leurs données vers Webflow, contenaient d'anciens systèmes de vérification SMS morts ou incomplets, ou affichaient une confirmation avant même de savoir si le lead avait été transmis.

Le travail réalisé a porté successivement sur :

1. `assurance-tns-pro-v2-adw` — Assurance TNS Pro.
2. `assurance-tns` — Assurance TNS.
3. `assurance/agriculteur` — Assurance Agriculteur.
4. `assurance-consultants-societe-conseil` — Consultants & Société de Conseil.
5. `assurance-consultant-societe-conseil-v2` — page wrapper iframe.
6. `solutions-agv-amr` — Solutions AGV & AMR.
7. L'ensemble du site contenu dans `ubpro_complet_foctionnel.rar` : 176 pages auditées et traitées selon leur catégorie.
8. L'ajout final de la vérification du numéro de téléphone par SMS sur les six livraisons principales, puis sur les formulaires éligibles de la v2 ubepro complète.

Toutes les PR #1 à #11 sont fusionnées dans `main`. Le dernier état intégré contient les dossiers fonctionnels, leurs ZIP individuels, le ZIP regroupant les six premiers projets et l'archive v2 complète du site ubepro.

# 3. PRINCIPES PERMANENTS À RESPECTER

## 3.1 Style de travail

- Répondre en français.
- Être direct, rapide et efficace, sans sacrifier les tests décisifs.
- Agir comme un expert senior : diagnostiquer la cause réelle avant de corriger.
- Ne jamais conclure qu'un formulaire fonctionne sur la seule base d'un test superficiel.
- Expliquer clairement ce qui a été vérifié, ce qui a été modifié, ce qui reste à faire côté production.

## 3.2 Préservation de chaque LP

- Ne jamais écraser une landing page avec une autre (« sans écraser une »).
- Chaque LP garde son design, ses contenus, ses questions, son identité commerciale et son identité CRM.
- Le traitement fonctionnel doit s'adapter au formulaire réel de la page.
- Ne modifier le CSS que de manière minimale et justifiée lorsque c'est nécessaire.
- L'image hero ne doit être remplacée que si elle est hors sujet, manifestement incohérente ou de mauvaise qualité pour le thème. Si elle est déjà pertinente, la conserver.
- Les corrections SEO doivent rester légères et sans impact visuel : `lang="fr"` , H1 unique lorsque c'est sûr, preload d'image hero lorsque pertinent.

## 3.3 Objectif fonctionnel commun

Le standard atteint pour les formulaires de capture est le suivant :

- soumission autonome, sans dépendance à l'hébergement Webflow ;
- réception du contact et des réponses au questionnaire ;
- double livraison en parallèle vers le CRM HelloLead et Google Sheets ;
- confirmation affichée uniquement si au moins une livraison réussit ;
- validations e-mail, code postal et téléphone ;
- honeypot anti-spam silencieux ;
- événement GTM enrichi ;
- capture du tracking d'URL, filtrée par le consentement cookies ;
- APIs de contrôle qualité dans la console ;
- documentation de configuration et rapport de modifications ;
- tests automatisés avec le parcours réel de soumission ;
- génération d'un ZIP complet livrable.

## 3.4 Évolution importante concernant le SMS

Au début du projet, l'utilisateur voulait supprimer toute confirmation par code SMS et simplement récupérer le contact. Cette règle a été appliquée aux premières versions.

Plus tard, l'utilisateur a explicitement demandé l'ajout d'une vérification du numéro par SMS sur les six livraisons principales, puis son application au contenu du `.rar`.

**La règle actuelle est donc : la vérification SMS fait partie du standard final.**

- Sur les formulaires avec téléphone, l'envoi du lead est bloqué tant que le numéro n'est pas vérifié.
- Le numéro doit être un mobile français de 10 chiffres commençant par 06 ou 07.
- Après vérification réussie, `telephone_verifie=oui` est transmis au CRM, à Google Sheets et à GTM.
- Ne pas revenir à l'ancien modèle « sans SMS », sauf demande explicite contraire de l'utilisateur.

## 3.5 Règles Git et GitHub

- Toujours travailler sur la branche de session fournie par Arena dans la discussion active.

- Ne pas renommer la branche de session.
- Ne pousser que vers cette branche, sauf instruction système contraire.
- N'ouvrir une PR que si l'utilisateur le demande.
- Ne fusionner une PR que si l'utilisateur le demande.
- Toujours faire `git fetch origin` avant de vérifier l'état, créer une PR ou comparer l'historique.
- `main` a été recréée plusieurs fois par upload GitHub, produisant des historiques non liés. Avant chaque PR :
  - vérifier `git merge-base HEAD origin/main` ;
  - comparer les arbres et confirmer que les contenus communs sont identiques ;
  - si l'historique est non lié et que seul le nouvel apport diffère, réunir par `git merge origin/main --allow-unrelated-histories` ;
  - vérifier ensuite que le diff de PR ne contient que les fichiers attendus.
- Utiliser `--body-file` pour les longues descriptions de PR afin d'éviter les erreurs d'échappement shell avec les backticks.
- Pour fusionner : `gh pr merge <numéro> --merge`, sans pipe ni opérateur qui masquerait le code retour, puis vérifier séparément l'état et `origin/main`.

# 4. ÉTAT FINAL DU DÉPÔT

## 4.1 Dossiers principaux

Le dépôt contient notamment :

- `assurance-consultant-conseil-v2/` — LP de référence fonctionnelle d'origine.
- `assurance-tns-pro-v2-adw/` — Assurance TNS Pro traitée.
- `assurance-tns/` — Assurance TNS traitée.
- `assurance/agriculteur/` — Assurance Agriculteur traitée.
- `assurance-consultants-societe-conseil/` — Consultants & Société de Conseil traitée.
- `assurance-consultant-societe-conseil-v2/` — wrapper iframe traité.
- `solutions-agv-amr/` — Solutions AGV & AMR traitées.
- `assurance/` — dossier parent historique pour `agriculteur` dans le dépôt.
- `ubpro_complet_foctionnel.rar` — archive source d'origine du site complet.
- `ubpro_complet_fonctionnel-v2.zip` — version 2 traitée du site complet.
- `RAPPORT-UBPRO-V2.md` — rapport de la v2 complète.
- `LES-6-PROJETS-FONCTIONNELS-FINAL.zip` — regroupement des six premiers projets.
- `LISEZ-MOI-6-PROJETS.md` — récapitulatif des six premiers projets.

## 4.2 Archives livrables connues

| Fichier | Taille approximative | Contenu |
|---|---|---|
| assurance-tns-pro-v2-adw-FONCTIONNEL.zip | 5,2 Mo | Assurance TNS Pro |
| assurance-tns-FONCTIONNEL.zip | 3,7 Mo | Assurance TNS |
| agriculteur-FONCTIONNEL.zip | 5,2 Mo | Assurance Agriculteur |
| assurance-consultants-societe-conseil-FONCTIONNEL.zip | 5,7 Mo | Consultants & Société de Conseil |
| assurance-consultant-societe-conseil-v2-FONCTIONNEL.zip | 3,3 Mo | Wrapper iframe |
| solutions-agv-amr-FONCTIONNEL.zip | 12 Mo | Solutions AGV & AMR |
| LES-6-PROJETS-FONCTIONNELS-FINAL.zip | 34 Mo | Les six projets regroupés |
| ubpro_complet_fonctionnel-v2.zip | 52 Mo | Les 176 pages du site ubepro traitées |

## 4.3 Liens de téléchargement direct

Dépôt :

https://github.com/manajahelvin-ux/PorojetMigration

Dépôt complet, branche `main`, en ZIP :

https://github.com/manajahelvin-ux/PorojetMigration/archive/refs/heads/main.zip

ZIP final des six premiers projets :

https://github.com/manajahelvin-ux/PorojetMigration/raw/main/LES-6-PROJETS-FONCTIONNELS-FINAL.zip

Site ubepro complet v2 :

https://github.com/manajahelvin-ux/PorojetMigration/raw/main/ubpro_complet_fonctionnel-v2.zip

Rapport ubepro v2 :

https://github.com/manajahelvin-ux/PorojetMigration/blob/main/RAPPORT-UBPRO-V2.md

# 5. HISTORIQUE DES TRAVAUX ET DÉCISIONS

## 5.1 Assurance TNS Pro — assurance-tns-pro-v2-adw

- LP Webflow/lp2site avec formulaire multi-étapes Formly.
- Image hero initiale hors sujet (salon de beauté) remplacée par `img/hero-tns-pro.jpg` .
- Ancienne vérification SMS initialement retirée à la demande de l'utilisateur.
- Cause racine du « Oops! » identifiée : Formly appelle la soumission via jQuery `$form.submit()` , ce qui déclenchait le gestionnaire Webflow `.w-form form`, lequel tentait un envoi AJAX vers Webflow et échouait hors hébergement Webflow.
- Correction durable : wrapper `.w-form` remplacé par `.form-autonome` , marge compensée, nouveau `lead-handler.js` autonome.
- Identité CRM : `ADW_LP_ASSURANCE_TNS_PRO` .
- Source de secours : `LP TNS PRO`.
- Version fonctionnelle ensuite alignée sur la LP de référence, puis vérification SMS réintroduite lors de la phase finale.

## 5.2 Assurance TNS — assurance-tns

- Même traitement fonctionnel, mais identité et design propres conservés.
- Le hero réel est `.header-tns` , fond bleu nuit plein. Ne pas supposer qu'il s'agit d'un hero photo.
- Une ancienne image spa orpheline `img/hero-background.jpg` a été supprimée car la règle CSS associée était morte.
- `lang="fr"` , H1 unique.
- Identité CRM : `ADW_LP_ASSURANCE_TNS` .
- Source de secours : `LP TNS`.
- `form_name` : `Assurance-TNS-Form` .
- Domaine cible relevé : `assurance-tns.lp.comparateurpro.fr` .

## 5.3 Assurance Agriculteur — assurance/agriculteur

- Formulaire `wf-form-assurance-agriculteur-formly` , huit étapes.
- Questions : contrats agricoles en cases à cocher `reponse_1_1` à `reponse_1_5` , texte libre `reponse_2` , salariés `reponse_3` , auto-entrepreneur `reponse_4` , société existante `reponse_5` .
- Champ micro-entrepreneur : `reponse_4` .
- Hero d'origine pertinent : agriculteur en tracteur devant un vignoble (`Cepahges-1200x630-1.avif` ), conservé.
- Identité CRM : `ADW_LP_ASSURANCE_AGRICULTEUR` .
- Source de secours : `LP AGRICULTEUR` .
- `form_name` : `assurance-agriculteur-formly` .

## 5.4 Consultants & Société de Conseil — assurance-consultants-societe-conseil

- Clone fonctionnel proche de TNS Pro.
- Hero d'origine identique au visuel salon de beauté hors sujet ; remplacé par `img/hero-consultants.jpg` .
- Aucun H1 au départ : le titre du hero a été converti en H1 sans changer le rendu.
- Identité CRM : `ADW_LP_ASSURANCE_CONSULTANTS_SOCIETE_CONSEIL` .
- Source de secours : `LP CONSULTANTS SOCIETE CONSEIL` .
- `form_name` fonctionnel : `Assurance-Consultants-Societe-Conseil-Form` .
- Champ micro-entrepreneur : `reponse_3` .

## 5.5 Wrapper iframe — assurance-consultant-societe-conseil-v2

Ce dossier n'est pas une LP à formulaire. C'est une page wrapper contenant une iframe vers une LP hébergée.

Traitement effectué :

- ajout de GTM `GTM-53SH7D7F` après le Consent Mode par défaut déjà présent ;
- balise GTM `noscript` ;
- `lang="fr"` ;
- suppression d'un bundle `script.js` inutile d'environ 1,5 Mo ;
- conservation de `style.css` , nécessaire à `.lp-assurance-frame` ;
- création puis extension de `bridge.js` ;
- ajout du relais SMS parent selon le protocole de la LP de référence ;
- double protocole conservé : format référence et ancien format des LP du dépôt ;
- documentation spécifique iframe et SMS.

Point critique restant : la LP hébergée dans l'iframe doit accepter l'origine parente réelle du wrapper. Historiquement, la LP de référence n'acceptait que `https://www.comparateurpro.fr` . L'API SMS doit aussi autoriser l'origine du wrapper en CORS, car le parent exécute les POST SMS depuis son domaine.

## 5.6 Solutions AGV & AMR — solutions-agv-amr

- LP déjà présentée comme fonctionnelle, avec un gestionnaire intégré dans un gros `script.js` .
- Audit : aucun SMS, CRM et Sheets présents, mais « Merci » affiché avant les envois, aucune validation bloquante, filtragede consentement incomplet, Sheets en `no-cors` déclaré confirmé sans vérification.
- Le bloc intégré a été remplacé par le handler standard séparé, à identité inchangée.
- Identité CRM : `ADW_LP_SOLUTIONS_AGV_AMR` .
- Source de secours : `ADW`.
- `form_name` : `Email-Form-AGV` .
- Ce `formId` était considéré comme déjà en production : aucune nouvelle inscription CRM n'était requise pour cette LP.
- Ancien bandeau hero abstrait remplacé par `img/hero-agv-amr.jpg` : entrepôt automatisé avec AMR.
- Page passée de 19 H1 à un H1 unique, les autres titres devenant des H2.

## 5.7 Vérification SMS ajoutée aux six livraisons principales

La PR #9 a ajouté :

- `verification-sms.js` sur les cinq pages à formulaire ;
- blocage de `submitLead()` sans vérification ;
- `telephone_verifie` dans les données CRM, Sheets et GTM ;
- relais SMS complet dans le wrapper ;
- `docs/VERIFICATION-SMS.md` dans chaque dossier ;
- régénération des six ZIP fonctionnels.

Tests de cette phase : 26 scénarios réussis — 20 sur les cinq formulaires et 6 sur le wrapper.

## 5.8 Site ubepro complet v2 — contenu du .rar

Archive source : `ubpro_complet_foctionnel.rar` , environ 49 Mo, 3 330 fichiers, 176 pages `index.html` .

### Audit initial

- Toutes les pages étaient déjà en `lang="fr"` .
- 128 pages sans H1, 23 avec un H1, 25 avec plusieurs H1.
- 88 pages sans identifiant GTM détecté dans le HTML initial.

- 175 bundles contenaient un gestionnaire intégré minifié commun.
- Ce gestionnaire interceptait tous les formulaires, y compris panier, recherche et mot de passe.
- Il affichait le succès avant les envois et ignorait les erreurs réseau.
- Il n'y avait ni validation finale, ni honeypot réellement bloquant, ni filtrage complet par consentement, ni vérification SMS.
- Le `formId` CRM correspondait simplement à l'id ou au nom HTML du formulaire.

### Classification finale

| Catégorie | Nombre | Signification | Traitement |
|---|---|---|---|
| A_TEL | 97 | Formulaire coordonnées avec téléphone | Traitement complet + SMS |
| B_MAIL | 2 | Formulaire coordonnées sans téléphone | Traitement complet sans SMS |
| C_QUALIF | 56 | Qualification seule, sans coordonnées | Inchangées fonctionnellement |
| PAGE | 21 | Pages, TYP, commerce, utilitaires, portails | Gestionnaire intégré retiré |

Les pages B sont `fuso-v2` et `solutions-agv` .

### Traitement v2

- 119 bundles débarrassés du gestionnaire intégré.
- 99 `lead-handler.js` instanciés.
- 97 modules `verification-sms.js` installés.
- 26 pages lead ont reçu `GTM-53SH7D7F` .
- H1 corrigé prudemment sur 42 pages.
- Environ 1 250 champs cachés de tracking ajoutés.
- Résolution des champs par nom normalisé : minuscules, accents supprimés, espaces/tirets/underscores ignorés, suffixe numérique toléré. Exemples : `telephone-2` , `Nom 2`, `Email` , `Code Postal 2`.
- Message « Merci » créé en secours lorsqu'aucun bloc `.w-form-done` n'existe.
- Catégorie C volontairement inchangée : formulaires de qualification dans un tunnel en deux temps ; la capture a lieu plus tard sur des formulaires coordonnées traités.
- Images conservées : aucun exemplaire de l'ancien visuel salon de beauté n'a été retrouvé dans le `.rar` , et chaque verticale possédait son visuel métier.
- Archive finale livrée en `.zip` , car la création d'un `.rar` nécessite WinRAR propriétaire indisponible dans l'environnement.

### Tests v2

- Banc jsdom + vrai jQuery sur les 99 formulaires traités.
- Résultat final : **99/99**.
- 315 contrôles `node --check` réussis.
- Un défaut réel découvert par les tests sur `ds-automobiles-nouvelle-gamme-pallas` : champ réel `telephone-2` , alors que les alias initiaux cherchaient notamment `Telephone 2`. Correction : résolution par nom normalisé dans le handler, le module SMS et le banc de test.

# 6. CONTRAT CRM HELLOLEAD

Endpoint :

```
https://mc.hellolead.pro/mon_compte/api/webflow/get_information.php
```

Enveloppe JSON :

```
{
  "payload": {
    "formId": "...",
    "id": "UUID",
    "data": {}
  }
}
```

Modes prévus par la référence et les handlers :

- `nocors` : `sendBeacon` avec Blob `text/plain`, repli `fetch no-cors`. L'envoi est considéré transmis mais **non confirmé** applicativement.
- `cors` : `fetch` JSON avec timeout et exigence d'une réponse JSON indiquant `status: "ok"`.

Les livraisons CRM et Sheets sont lancées en parallèle. La confirmation est affichée si `crm.ok || sheet.ok`. Si les deux échouent, l'utilisateur voit une erreur claire, le formulaire reste visible et l'envoi peut être retenté. Il ne faut jamais réintroduire un faux « Merci » avant livraison.

# 7. CONTRAT GOOGLE SHEETS

Endpoint Apps Script utilisé par les projets traités :

```
https://script.google.com/macros/s/AKfycbwrbyCb9-cAmIRPGNvnb-aM-CkhQAerrkfP3_HSw-ewmThxsn4vbZt5daaq2aOnL01a1w/exec
```

Principes :

- corps plat en `URLSearchParams` ;
- transport CORS ;
- timeout de 15 secondes ;
- réponse JSON devant indiquer `status: "ok"` ;
- `solution` : en général la réponse principale nettoyée, avec particularité AGV ;
- `enjeu` : récapitulatif `autre_info_fj` nettoyé ;
- questions/réponses transmises lorsque présentes ;
- `website: ""` conservé comme champ de compatibilité/honeypot dans le payload feuille ;
- suppléments de tracking seulement s'ils sont non vides et autorisés par le consentement.

Particularité Solutions AGV & AMR du dépôt principal :

- `solution` = `reponse_3` en priorité (solutions cochées), sinon `reponse_1` ;
- `site` = `reponse_2` ;
- alias QA historiques `agvLead` et `ubeproLead` conservés dans l'instance principale.

Chaque dossier des six premiers projets contient un `docs/Code.gs` de référence pour créer ou adapter une feuille dédiée si nécessaire. Ne pas supposer qu'une nouvelle feuille est automatiquement configurée : toujours vérifier l'endpoint réellement présent dans le handler et la destination métier attendue.

# 8. FONCTIONNEMENT DU lead-handler.js

Les handlers actuels sont dérivés d'un standard commun, instancié par LP. Ne pas recopier aveuglément une instance sans adapter son identité et son sélecteur de formulaire.

Fonctions principales :

- sélection du formulaire réel de la LP ;
- interception de la soumission simulée par Formly via jQuery et des soumissions natives en capture ;
- suppression des anciens handlers conflictuels sur le formulaire ciblé lorsque nécessaire ;
- garde de dernière étape fondée sur la visibilité du bouton final ;
- construction des données par scan des questions/réponses lorsque le formulaire suit le schéma Formly ;
- résolution des champs d'identité par alias sur la v2 ubepro ;
- validations e-mail, code postal, téléphone et consentement du formulaire si les champs existent ;
- honeypot `website` créé ou réutilisé, sans envoi ni événement lead s'il est rempli ;
- capture du tracking (UTM, `ltm_*`, alias, hash, click IDs) avec persistance de session ;
- filtrage par consentement avant tout envoi d'informations de tracking ;
- envois CRM et Sheets en parallèle avec états réels ;
- événement `formulaire_lead` ou `formulaire_micro_entrepreneur` ;
- sauvegarde locale utile aux pages de remerciement et redirection `ltm_typ` lorsque prévue ;
- APIs QA figées.

APIs utiles dans la console navigateur :

```
lpLead.preview()
lpLead.googleSheetPreview()
lpTracking.inspect()
smsVerification.estVerifie()
smsVerification.numeroVerifie()
```

Sur certaines anciennes instances ou compatibilités ubepro, les alias suivants peuvent exister : `ubeproLead`, `agvLead`, `tnsLead`, `ubeproTracking`, `agvLead`, `agvTracking`. Toujours vérifier dans le fichier réel avant de promettre une API précise.

Clés de stockage fréquentes :

- `lp_tracking_v1`
- `lp_tracking_page_url_v1`
- `lp_public_landing_url_v1`

Le site ubepro complet v2 reprend le même standard, avec des adaptations génériques supplémentaires : alias de champs normalisés, boutons input/lien, et message de confirmation créé si la page n'a pas de bloc succès.

# 9. TRACKING ET CONSENTEMENT

Règles à maintenir :

- Les UTM, `ltm_*`, `page_url`, referrer nettoyé et informations assimilées analytics ne sont inclus que si le consentement analytics est accordé.
- Les click IDs (`gclid`, `msclkid`, `fbclid`, `gbraid`, `wbraid`, `ttclid`) et `user_data` ne sont inclus que si le consentement marketing est accordé.
- Sans consentement approprié, utiliser la source de secours propre à la LP, jamais une valeur placeholder comme `SOURCE PAR DEFAUT`.
- La bannière des LP assurance utilise le cookie JSON `user_cookie_preferences`.
- Les pages ubepro peuvent aussi utiliser `ubeproCookiePreferencesV1` dans `localStorage`.
- Toujours vérifier le mécanisme réel de bannière dans le bundle de la page avant d'adapter le lecteur de consentement.
- L'événement GTM ne doit pas fuiter de click IDs ni de données personnelles sans consentement marketing.
- Deux conteneurs GTM coexistent dans le périmètre historique : `GTM-53SH7D7F` et `GTM-WMVZJKS`. Ne pas remplacer un conteneur existant sans raison métier. Le modèle utilisé pour les pages sans conteneur a été `GTM-53SH7D7F`.

URL de test type :

```
?utm_source=google&utm_medium=cpc&utm_campaign=test&gclid=TEST123&ltm_source=ADW_TEST&ltm_user=42
```

Après consentement analytics, la source peut devenir `ADW_TEST`. Le `gclid` ne doit apparaître qu'avec le consentement marketing.

# 10. VÉRIFICATION SMS — CONTRAT COMPLET

Endpoint :

```
https://mc.hellolead.pro/mon_compte/api/smspartner/envoi_code_verification.php
```

## 10.1 Envoi du code

Requête `POST` avec `URLSearchParams` ou `FormData`, jamais de JSON brut :

```
action=envoyer_code
telephone=0607080910
```

Le numéro est normalisé puis doit satisfaire :

```
^0[67]\d{8}$
```

Après succès :

- afficher le champ du code ;
- vider l'ancien code saisi ;
- donner le focus au champ ;
- afficher « Code envoyé par SMS » ;
- désactiver le renvoi et afficher un compte à rebours de 120 secondes ;
- informer que le code comporte 6 chiffres, expire après 10 minutes, est à usage unique et qu'un nouveau code remplace l'ancien.

## 10.2 Vérification du code

Requête :

```
action=verifier_code
telephone=0607080910
code=123456
```

Après succès :

- enregistrer le numéro vérifié ;
- positionner le champ caché `telephone_verifie` à `oui` ;
- masquer le champ code et le bouton d'envoi ;
- afficher clairement `✓ Numéro vérifié` ;
- autoriser la soumission du lead pour exactement ce numéro ;
- si l'utilisateur modifie le numéro, réinitialiser l'état vérifié et demander un nouveau code.

## 10.3 Réponses et erreurs

Toujours tenter de lire le JSON même lorsque le statut HTTP est en erreur. Le message JSON (`message` ou `error`) est prioritaire sur les messages par défaut.

| Statut | Signification | Comportement attendu |
|---|---|---|
| 200 | Succès, sauf indicateur d'échec dans le JSON | Continuer le parcours |
| 422 | Numéro ou code invalide | Afficher le message API |
| 404 | Aucun code envoyé | Inviter à demander un code |
| 409 | Code déjà utilisé | Demander un nouveau code |
| 410 | Code expiré | Demander un nouveau code |
| 429 | Demande trop rapide | Extraire le délai restant et synchroniser le compte à rebours |
| 502 | Échec d'envoi SMS | Inviter à réessayer |
| Réseau/CORS | Requête impossible | Message générique de connexion |

Champs de délai 429 déjà pris en charge : `retry_after` , `retryAfter` , `delai` , `delai_restant` , `restant` , `remaining` , `seconds_remaining` , puis nombre extrait du message, puis valeur par défaut de 120 secondes.

## 10.4 Blocage du lead

Dans `submitLead()` :

- si un champ téléphone existe mais que `window.smsVerification` est absent, bloquer avec un message de disponibilité ;
- si le numéro normalisé n'est pas celui qui a été vérifié, bloquer ;
- si le numéro n'est pas un mobile 06/07, afficher le message spécifique de mobile requis ;
- ne transmettre aucune donnée au CRM ou à Sheets lorsque la vérification n'est pas acquise.

## 10.5 CORS

Les appels directs partent du navigateur depuis le domaine public de chaque LP. L'API doit autoriser chaque origine de déploiement. Un test local avec mocks ne prouve pas le CORS réel : un test SMS réel sur domaine déployé reste obligatoire avant production.

# 11. WRAPPER IFRAME ET RELAIS SMS

Le wrapper `assurance-consultant-societe-conseil-v2` utilise `bridge.js`.

## Protocole de référence

- Canal : `ubepro-assurance-v1`.
- L'enfant envoie `bridge-ready`.
- Le parent génère un nonce et répond `bridge-init`.
- L'enfant envoie `consent-state` avec ses choix analytics/marketing.
- Le parent répond `landing-url` avec l'URL publique filtrée selon le consentement.
- Pour le SMS, l'enfant envoie `sms-request` avec `requestId`, `telephone`, action et éventuellement `code`.
- Le parent valide localement le mobile et le format du code.
- Le parent appelle l'API SMS en `URLSearchParams`.
- Le parent répond `sms-response` avec `requestId`, statut HTTP, booléen `ok` et objet JSON lu même en erreur.
- Toute origine, source de fenêtre et nonce doivent être strictement vérifiés.

Le parent conserve aussi l'ancien format des LP du dépôt (`bridge: 'ubepro-assurance-v1'`, requêtes `consent-state-request` et `landing-url-request`) au cas où l'iframe serait repointée vers l'une des LP traitées.

Conditions réelles d'efficacité :

1. La LP enfant doit autoriser l'origine du wrapper comme parent.
2. L'API SMS doit autoriser l'origine du wrapper en CORS.
3. Le wrapper doit être servi en HTTPS.
4. La liste `allowedParents` de `bridge.js` doit contenir l'origine réelle de déploiement.

## 12. IDENTITÉS CRM DES SIX PREMIERS PROJETS

| Dossier | formId | Source de secours | Remarque |
|---|---|---|---|
| assurance-tns-pro-v2-adw | ADW_LP_ASSURANCE_TNS_PRO | LP TNS PRO | À enregistrer/vérifier côté CRM |
| assurance-tns | ADW_LP_ASSURANCE_TNS | LP TNS | À enregistrer/vérifier côté CRM |
| assurance/agriculteur | ADW_LP_ASSURANCE_AGRICULTEUR | LP AGRICULTEUR | À enregistrer/vérifier côté CRM |
| assurance-consultants-societe-conseil | ADW_LP_ASSURANCE_CONSULTANTS_SOCIETE_CONSEIL | LP CONSULTANTS SOCIETE CONSEIL | À enregistrer/vérifier côté CRM |
| assurance-consultant-societe-conseil-v2 | Aucun propre | Sans objet | Wrapper iframe et relais SMS |
| solutions-agv-amr | ADW_LP_SOLUTIONS_AGV_AMR | ADW | Déjà considéré en production |

Pour les 99 formulaires de la v2 ubepro, les identités sont dérivées du slug du dossier selon la convention `ADW_LP_<SLUG>` et listées dans l'annexe de `RAPPORT-UBPRO-V2.md`. Toujours lire le `formId` réel dans le `lead-handler.js` avant toute action CRM ; ne pas se fier à une déduction.

# 13. RÈGLES DE TEST NON NÉGOCIABLES

## 13.1 Chemin décisif

Le chemin qui compte n'est pas un simple événement `submit` natif : Formly déclenche l'envoi par jQuery, historiquement via le clic final puis `$form.submit()` .

Les tests doivent donc :

- utiliser jsdom ;
- charger le vrai jQuery ;
- extraire le vrai formulaire de la page traitée ;
- charger `lead-handler.js` puis `verification-sms.js` dans l'ordre réel ;
- simuler l'envoi avec la soumission jQuery ;
- mocker `fetch` et `navigator.sendBeacon` ;
- inspecter les corps réellement envoyés, y compris les Blob `sendBeacon` de manière asynchrone.

Ne pas reproduire l'erreur historique de la PR #2, dont les tests insuffisants avaient produit une fausse confiance.

## 13.2 Environnement de test

- Le dossier `/tmp/test-form` peut être effacé entre sessions : le recréer à chaque nouvelle session.
- Installer `jsdom` et **épingler jquery@3.7.1**. jQuery 4 casse l'usage `require('jquery')(window)` dans ce contexte Node/jsdom.
- jsdom n'a pas de layout : simuler `Element.prototype.getClientRects` lorsque le handler utilise la visibilité du bouton final pour détecter la dernière étape.
- Positionner le consentement via le cookie `user_cookie_preferences` et, selon la LP, les clés localStorage de sa bannière.
- Attention : le champ e-mail des formulaires standards s'appelle souvent `mail`, pas `email`.
- Cocher le radio `consentement=oui` lorsqu'il existe.
- Les noms réels `name` doivent être inspectés, pas seulement les `data-name` . Exemple : `telephone-2` n'est pas `Telephone 2`.
- Le corps CRM envoyé par beacon peut être un Blob : utiliser `await body.text()` avant `JSON.parse` .

## 13.3 Scénarios minimaux pour un nouveau formulaire

1. Soumission refusée sans vérification SMS, aucun appel CRM/Sheets.
2. Envoi du code avec les bons paramètres `envoyer_code` et numéro normalisé.
3. Mauvais code ou erreur 422 : message affiché, numéro non vérifié.
4. Code correct : badge de vérification et `estVerifie()` vrai.
5. Soumission ensuite acceptée : CRM avec bon `formId`, `telephone_verifie=oui` , Sheets transmis.
6. Réponse 429 : délai restant affiché et bouton de renvoi désactivé avec compteur.
7. Réponse 410 : message d'expiration.
8. Livraisons CRM et Sheets en échec : pas de faux « Merci », erreur visible, nouvel essai possible.
9. Honeypot rempli : aucun envoi.
10. Sans consentement : aucun click ID et source de secours propre.
11. Avec consentement complet : tracking et événement GTM enrichis.

## 13.4 Limites des tests

Les tests jsdom utilisent des mocks. Ils ne prouvent ni le CORS réel de l'API SMS, ni l'enregistrement CRM d'un formId, ni l'autorisation d'origine parente par une LP externe hébergée, ni la réception d'un vrai SMS. Ces points exigent un test sur domaine déployé.

## 14. ERREURS DÉJÀ RENCONTRÉES À NE PAS REFAIRE

- Chercher `.submit()` en clair dans Formly est inefficace : le code est obfusqué et utilise des appels entre crochets. En cas de doute, décoder le tableau de chaînes, son décodeur et l'IIFE de rotation avec ses arguments d'invocation.
- Une capture native de l'événement `submit` ne suffit pas forcément : jQuery peut être le vrai chemin.
- Retirer les handlers du formulaire peut aussi supprimer un handler d'inputmask ; ce compromis a été accepté car la soumission est virtuelle, mais il faut y penser en cas de régression de masque.
- Les fichiers Webflow ont souvent des fins de ligne CRLF ou mixtes. Les patchs Python doivent lire en `newline=''` et tolérer CRLF.
- Dans un commentaire JS en bloc, éviter toute séquence `*/` accidentelle, par exemple dans `utm_*/ltm_*` , qui ferme prématurément le commentaire et provoque une `SyntaxError` .
- Ne pas paralléliser une écriture de fichier et le contrôle qui doit lire immédiatement le résultat : risque de course. Exécuter les contrôles après l'écriture.
- Ne pas lancer `pkill -f "http.server <port>"` avec une chaîne présente dans la commande elle-même : le shell peut se tuer. Préférer `fuser -k <port>/tcp` ou le gestionnaire de processus.
- `git fetch origin main` seul a déjà laissé `origin/main` obsolète dans ce workflow. Utiliser `git fetch origin` complet.
- Ne pas supposer qu'une page a un hero photo. Exemple : `assurance-tns` utilise un hero bleu nuit plein.
- Ne pas interpréter `data-name` comme le vrai attribut `name` d'un champ.
- Ne pas classifier un formulaire uniquement à son nom de classe contenant « commerce » : `form-commerce` peut être un formulaire métier, alors que Webflow Commerce utilise `w-commerce...` .
- Ne pas faire confiance aux gestionnaires intégrés qui affichent la réussite avant d'attendre les livraisons.
- Ne pas considérer un transport `no-cors` comme une confirmation applicative de réception.

## 15. PROCÉDURE POUR AJOUTER OU TRAITER UNE NOUVELLE LP

1. Inspecter le dossier et identifier `index.html` , `script.js` , `style.css` , les images et les formulaires.
2. Inventorier chaque formulaire : id, nom, classe, champs réels `name`, questions/réponses, téléphone, e-mail, code postal, consentement, bouton final, messages `.w-form-done` et `.w-form-fail` .
3. Vérifier le bundle : chemin de soumission réel, Webflow AJAX, gestionnaire intégré, anciens SMS Axialys/SMS Partner, GTM, bannière de consentement et mécanisme de stockage.
4. Classer la page : lead avec téléphone, lead sans téléphone, qualification seule, page de contenu, tunnel commerce/utilitaire, wrapper iframe.
5. Décider si le design et le hero doivent rester inchangés. Ne remplacer un hero que s'il est manifestement hors sujet ou inadapté.
6. Sauvegarder les fichiers sources avant patch.
7. Si le wrapper `.w-form` capte encore le formulaire, le remplacer par `form-autonome` et compenser uniquement la marge nécessaire.
8. Retirer ou neutraliser l'ancien gestionnaire qui entre en conflit, de manière chirurgicale et vérifiable.
9. Instancier `lead-handler.js` avec l'identité propre de la LP : formId, source de secours, form_name, sélecteur réel, champ micro éventuel, version.
10. Compléter les champs cachés de tracking avec leurs `id`, sans dupliquer ceux qui existent.
11. Ajouter `verification-sms.js` si un téléphone exploitable existe et bloquer l'envoi sans vérification.
12. Ajouter GTM seulement s'il est absent et nécessaire à la mesure de la LP ; ne pas écraser un conteneur existant.
13. Appliquer uniquement les corrections H1 sûres et sans effet visuel.
14. Ajouter la documentation propre à la LP et son rapport de modifications.
15. Exécuter les syntaxes JS, les contrôles statiques et le banc jsdom complet.
16. Servir localement et vérifier les ressources en HTTP 200.
17. Générer le ZIP complet du dossier fini.
18. Commit sur la branche de session, push uniquement vers cette branche.
19. Présenter le ZIP ou le livrable principal.
20. N'ouvrir et ne fusionner une PR que sur demande explicite.

## 16. CE QUI RESTE À FAIRE CÔTÉ PRODUCTION

Les éléments suivants ne pouvaient pas être validés uniquement dans le sandbox :

1. **CRM HelloLead** : enregistrer ou valider tous les nouveaux `formId`, notamment ceux des six premiers projets hors AGV déjà en production et les 99 de la v2.
2. **Lead réel par famille** : vérifier sa présence dans le CRM et dans la feuille Google, avec les bonnes colonnes et la bonne source.
3. **GTM** : utiliser le mode Aperçu avec consentement accepté, vérifier événements, variables et `user_data` uniquement après consentement marketing.
4. **CORS SMS** : autoriser chaque origine publique de déploiement côté API SMS.
5. **SMS réel** : tester l'envoi, la réception, l'expiration, l'usage unique, le renvoi à 120 secondes et les erreurs.
6. **Wrapper iframe** : faire accepter l'origine réelle du wrapper par la LP enfant hébergée et par l'API SMS.
7. **Tunnel ubepro complet** : tester une page de qualification de catégorie C jusqu'au formulaire coordonnées correspondant, puis SMS et CRM.
8. **Cache navigateur** : tester chaque archive dans un nouveau dossier ou une navigation propre pour éviter d'anciens JS en cache.
9. **Origines et domaines finaux** : vérifier les URL publiques, `publicLandingPath` si la LP est affichée en iframe, et l'URL de landing transmise par le parent.
10. **Conteneurs GTM multiples** : confirmer métier la coexistence de `GTM-53SH7D7F` et `GTM-WMVZJKS` sur le site ubepro complet.

## 17. ARBORESCENCE SIMPLIFIÉE FINALE

```
PorojetMigration/
├── assurance-consultant-conseil-v2/       Référence fonctionnelle d'origine
├── assurance-tns-pro-v2-adw/              Projet 1
├── assurance-tns/                         Projet 2
├── assurance/
│   └── agriculteur/                       Projet 3
├── assurance-consultants-societe-conseil/ Projet 4
├── assurance-consultant-societe-conseil-v2/ Projet 5, wrapper
├── solutions-agv-amr/                     Projet 6
├── LES-6-PROJETS-FONCTIONNELS-FINAL.zip
├── LISEZ-MOI-6-PROJETS.md
├── ubpro_complet_foctionnel.rar           Source du site complet
├── ubpro_complet_fonctionnel-v2.zip       Site complet traité
├── RAPPORT-UBPRO-V2.md
└── les ZIP individuels *-FONCTIONNEL.zip
```

À l'intérieur de l'archive v2, le dossier racine est `ubpro_complet_fonctionnel_v2/` et contient les 176 pages ainsi que `RAPPORT-UBPRO-V2.md` .

# 18. HISTORIQUE GIT RÉSUMÉ

| PR | Objet |
|---|---|
| #1 | Premier traitement TNS Pro : hero, nettoyage SMS initial, ZIP |
| #2 | Tentative insuffisante de capture native — le « Oops! » persistait |
| #3 | Cause racine Formly/jQuery/Webflow corrigée, handler autonome |
| #4 | Câblage CRM, Sheets, GTM et tracking |
| #5 | Parité fonctionnelle TNS Pro avec la référence et ZIP FONCTIONNEL |
| #6 | Assurance TNS FONCTIONNEL avec identité préservée |
| #7 | Agriculteur, Consultants et wrapper iframe |
| #8 | Solutions AGV & AMR mises au standard |
| #9 | Vérification SMS sur les six livraisons principales |
| #10 | ZIP final regroupant les six projets |
| #11 | Site ubepro complet v2, 176 pages |

Toutes ces PR sont fusionnées. Ne pas rouvrir ou refaire leur travail sans demande explicite et diagnostic nouveau.

## 19. CONSIGNES POUR LE PROCHAIN ASSISTANT

À réception de ce document dans une nouvelle discussion :

1. Lire ce fichier entièrement avant d'agir.
2. Vérifier la branche de session indiquée par Arena dans la nouvelle conversation ; elle peut être différente de `arena/e1f685c7-porojetmigration` .
3. Exécuter `git status` , `git fetch origin` , vérifier le dernier commit et l'état de `origin/main` .
4. Ne pas supposer qu'un fichier décrit ici existe encore sous le même nom : contrôler le dépôt réel.
5. Si l'utilisateur demande le même principe sur une nouvelle LP, suivre la procédure de la section 15.
6. Si l'utilisateur signale un bug, diagnostiquer le vrai chemin de soumission avant de modifier quoi que ce soit.
7. Si le sujet touche aux formulaires déjà livrés, lire d'abord le `lead-handler.js` , le `verification-sms.js` , le `docs/` et le `RAPPORT-MODIFICATIONS.md` du dossier concerné.
8. Tester avant d'annoncer une correction.
9. Générer un nouveau ZIP versionné plutôt que modifier silencieusement un ancien livrable lorsque l'utilisateur demande une nouvelle version.
10. Ne demander aucun mot de passe, token GitHub, code 2FA ou secret en discussion.

## 20. PHRASE DE SYNTHÈSE À RETENIR

Le projet est passé de landing pages qui semblaient fonctionner à un parc statique réellement autonome : les leads sont validés, le numéro mobile est vérifié par SMS, les envois CRM et Sheets sont contrôlés, le tracking respecte le consentement, les événements GTM sont structurés et chaque livraison est testée puis archivée — tout en préservant l'identité propre de chaque page.